Zaxonlite

Replicated SQLite on Multi-Paxos: WAL frames as the unit of consensus, a journal you can trust, and one implementation from an embedded durable node to role-aware clustered deployments

```
execute -> capture frames -> persist payload -> append
journal + fsync -> confirm -> committed -> acknowledge
```

Vikrant Rathore · Ronak Rathore

with the paxos-zig library and the `zaxon` command line

July 2026

About this book

This book documents Zaxonlite: an embeddable SQLite service replicated by the paxos-zig Multi-Paxos library, and the `zaxon` command line that hosts and drives it. It is written for three readers at once:

1. **the application developer**, who wants a durable SQL store with exactly-once write retry and knows exactly what is guaranteed;
2. **the operator**, who deploys `zaxon serve` alone or with voters and learners and needs the file formats, the recovery story, and the failure playbook;
3. **the engineer**, who wants to see how WAL-frame replication, a journal-authoritative storage design, and an explicit-effects Paxos core compose into a small, testable system.

The implemented vertical slice is exercised by unit tests, single-process durability tests, a three-process loopback cluster scenario with a SIGKILL failpoint, a CLI contract test, a C ABI smoke test, seeded property fuzzing, a soak run, role/gateway/adverse-network integration tests, and benchmarks. Protocol v4 adds mutually authenticated, sequenced HMAC framing and voter-certified learner commits. The 10,000-crash, 100-run, and 1-GiB stress targets are explicitly deferred; where this book states a current guarantee, the final chapter names its evidence.

> **Reading order**
>
> Chapters 1–3 explain what Zaxonlite is and the one idea it is built on (replicating committed WAL frames). Chapters 4–6 are the storage, cluster, and consistency deep dives. Chapters 7–9 are reference: operations, APIs, and on-disk/wire formats. Chapter 10 is the evidence.

Contents

1 The product

1.1 What Zaxonlite is

Zaxonlite is SQLite with a replicated, crash-consistent history. An application links one Zig library (or the C ABI), opens one data directory, and gets a full SQL database whose every committed write transaction is decided by Multi-Paxos and recorded in a checksummed journal before it is acknowledged. The same library and the same on-disk layout serve two deployments:

1. **one durable node**: a single process, no network, every write fsynced through the journal — an upgrade path for any application that outgrew plain SQLite's durability story;
2. **role-aware clusters over TCP**: one to nine configured voters run Paxos; witnesses vote without campaigning, standbys/read replicas learn chosen entries without changing quorum, and stateless gateways route clients.

The companion CLI is `zaxon`. It embeds a node directly (`--data`) or speaks the client RPC protocol to running servers (`--connect`), following leader redirects.

1.2 The one idea

Most replicated-SQLite systems replay SQL text on every node and hope the replicas compute identical results. Zaxonlite replicates **the bytes SQLite itself committed**: the page frames of the write-ahead log. The leader executes a transaction once, captures exactly the frames SQLite appended for it, and replicates those frames. Nondeterministic SQL — `random()`, `randomblob()`, dates, all of it — is safe by construction, because the decision being replicated is the page images, not the recipe that produced them.

> **Consequence: byte-identical replicas**
>
> Applying a committed frame payload is a deterministic page-write plus a truncate. Given the same base image and the same decided history, every member's database file converges to the same bytes. The verification suite checks this literally, with SHA-256 digests of `VACUUM INTO` images across all three cluster members.

1.3 User-visible guarantees

1. **Acknowledged means durable and decided.** A write is acknowledged only after its descriptor is committed by the configured voter quorum, its journal records are fsynced, and the slot has been applied locally.
2. **Exactly-once retry.** A client session executes sequence n at most once. Retrying the last sequence returns the recorded result — across process crashes, leader changes, and restarts — without executing SQL. Gaps and expired sequences fail without side effects.
3. **One-writer serializability.** All writes flow through the current leader, one replicated transaction at a time; a dependent slot is never proposed before its predecessor is chosen.
4. **Read levels you can name.** `any` reads locally (possibly stale on a follower and labeled as such); `leader` reads the leader's applied state; `linearizable` first completes a quorum read fence that performs no log append and no disk sync.
5. **The journal is the database.** The SQLite file is a materialized image; deleting it (or restoring a stale copy) converges back to the decided state from snapshot plus journal suffix.

1.4 Node types and scaling

`data-voter` proposes, votes, materializes SQLite, and serves SQL. `witness` votes and stores durable payloads but cannot campaign or serve SQL. `standby` and `read-replica` are non-voting chosen-log learners

with SQLite copies; only the standby is marked promotion-eligible. `gateway` is a byte-transparent router with no Paxos or SQLite state.
For voter set V, majority $q = \left\lfloor |V| \frac{1}{2} \right\rfloor + 1$. Learners are absent from V, so adding them does not change write quorum. The implementation profile bounds voters at nine but allocates the total registry at runtime. That is a scaling boundary, not a promise that millions of all-to-all sockets are practical.

1.5 Explicit non-goals

Zaxonlite does not do multi-master writes, cross-database transactions, SQL-visible replication controls, or automatic voter replacement. It bounds one epoch at 256 slots and one consensus group at nine voters; both are policy choices of the `ReplicatedLog` instantiation, not Paxos theorems.

2 Architecture

2.1 Components

One node owns one data directory and is assembled from small, separately tested parts:

Module	Responsibility
`node.zig`	The host: lifecycle, the write path ordering contract, effect handling, recovery, snapshots, epoch rollover, and the leader/follower split of the materialized image.
`journal.zig`	The framed, CRC-checksummed, append-only protocol journal; replay with torn-tail truncation and interior-corruption rejection.
`payload_store.zig`	Content-addressed immutable payloads under `payloads/aa/<hash>`, installed with write-temp/sync/link.
`wal.zig`	WAL frame capture from SQLite's `-wal` file and the deterministic offline page apply; the payload wire format.
`command.zig`	The fixed-size replicated descriptor (`TransactionBatch`) and the cumulative chain-hash formula.
`types.zig`	The one `ReplicatedLog` instantiation and canonical encodings of its entries and durable writes.
`sqlite.zig`	Narrow bindings over the pinned SQLite amalgamation: exactly the C API subset the product needs.
`server.zig` / `client.zig` / `wire.zig`	The TCP host behind `zaxon serve`, the RPC client, and the shared frame codec.
`capi.zig`	The C ABI (`libzaxonlite` + `zaxonlite.h`).

The Paxos core is the parent `paxos` library's `ReplicatedLog`: a bounded, allocation-free effect machine. The host never mutates protocol state directly; it calls `append`, `step`, `tick`, and consumes an explicit `Effects` batch whose contract **is** the durability ordering: persist `writesSlice()`, fsync, `confirmWritesDurable()`, only then send `messagesSlice()` and apply `committedSlice()`.

2.2 The replicated command

Paxos never carries transaction bytes. A committed slot holds a fixed-size descriptor:

```
TransactionBatch {
    database_id:       u128,   // cluster-wide database identity
    batch_id:          u128,   // random identity of this execution
    base_data_slot:    u64,    // previous data slot in the epoch
    base_chain_hash:   [32]u8, // cumulative history before this batch
    result_chain_hash: [32]u8, // cumulative history after this batch
    payload_hash:      [32]u8, // SHA-256 name of the frame payload
    payload_bytes:     u64,
    transaction_count: u32,
    frame_count:       u32,
}
```

The payload — header, per-transaction records, frame descriptors, raw page images — lives in the content-addressed store and is persisted **before** the descriptor is proposed, so no counted vote can ever name bytes that are not durable somewhere.

Chain hashes: O(1) history identity

`result_chain_hash = H(base_chain_hash, database_id, batch_id, base_data_slot, payload_hash, payload_bytes, transaction_count, frame_count)` with a domain-separated SHA-256. Two nodes with equal chain values have applied the same batches in the same order. Recovery, commit accounting, and `integrity-check` all validate the chain; a mismatch is fatal, never repaired silently.

2.3 The write path

Every write follows one ordering contract, identical in one-member and cluster deployments:

1. execute the SQL inside `BEGIN IMMEDIATE … COMMIT` on the leader's capture connection; the replicated session row and the `batch_id` marker are updated **inside the same SQLite transaction**;
2. capture the committed frames from the `-wal` file (the `wal_hook` count says exactly how many);
3. persist the payload in the content-addressed store (write, fsync, link);
4. append the descriptor through `ReplicatedLog`;
5. journal the resulting durable writes and fsync, then confirm;
6. only now may accept messages leave the process;
7. when the slot commits and carries **this** `batch_id`, acknowledge.

A failed SQL statement rolls back before capture; rolled-back transactions never advance the WAL hook count, so nothing about them is ever captured, proposed, or replicated.

2.4 The read path

The leader holds the only live SQLite connection (the capture connection); its applied state is the database. Followers keep the image materialized offline and open short-lived read connections per query. Reads are labeled: `any` may be stale on a follower; `leader` is the leader's applied state; `linearizable` completes a quorum fence first (chapter 6).

2.5 Sessions inside the state machine

Bounded client sessions are rows in a replicated table (`__zaxon_sessions`), updated inside the same captured transaction as the user's SQL. Exactly-once retry therefore needs no separate log: wherever the frames go, the session state goes, atomically. The `__zaxon_meta.batch_id` marker travels the same way and lets recovery prove the materialized image matches the last committed descriptor.

3 WAL-frame replication

3.1 Why frames, not SQL

Replaying SQL text on replicas requires every statement to be deterministic, every schema hook to fire identically, and every SQLite version to behave the same. WAL-frame replication removes the whole class: the leader's SQLite computes the transaction once, and consensus decides the resulting page images. dqlite and rqlite arrived at related designs by patching or wrapping SQLite; Zaxonlite's Phase-0 spike proved the capture can be done with **public, stable** SQLite interfaces only.

3.2 The capture technique (decided ADR)

The node runs one writer connection in WAL mode with automatic checkpoints disabled:

```
pragma journal_mode = wal;        -- append frames, never rewrite pages
pragma wal_autocheckpoint = 0;    -- checkpoints only at snapshots
pragma synchronous = normal;      -- journal fsync is the durability
sqlite3_wal_hook(db, hook, ...)   -- committed frame count per commit
```

After every commit, the hook reports the total number of committed frames in the WAL. The frames between the previous count and the new count are exactly this transaction's committed pages, and their bytes are read directly from the `current.db-wal` file: a 32-byte WAL header, then per-frame 24-byte headers (big-endian page number at offset 0, commit size at offset 4) followed by the raw page image. A rolled-back transaction never advances the hook count, so its frames are invisible to capture.

> **Why this is safe to read directly**
>
> Only the capture connection writes the WAL, checkpoints are disabled, and capture happens synchronously after `COMMIT` returns and before the next write begins. The `[from, to)` frame range is immutable by the time it is read. The one WAL subtlety that matters — cumulative frame checksums seeded per-WAL — is irrelevant because Zaxonlite never splices frames into another WAL; it applies pages offline.

3.3 The apply technique

A committed payload is applied **offline** — no SQLite connection open on the file:

1. for each frame, write the page image at `(page_number - 1) * page_size`;
2. truncate the file to the commit frame's database size in pages;
3. fsync.

Given the same base image and the same frames, this transition is deterministic and idempotent: applying any decided prefix again from any intermediate state converges to the same bytes. That single property powers recovery (rebuild from snapshot plus suffix), follower apply, and leadership-change resync, and makes crash timing across the whole apply path harmless.

3.4 The proof: byte-identical rebuild

The spike test — kept as a permanent unit test — runs DDL with triggers and indexes, DML, 20-KiB blobs, savepoints with partial rollbacks, `ALTER TABLE`, full rollbacks, and nondeterministic `randomblob()`/`random()` traffic on a live database while capturing every committed transaction. It then applies the captured payloads offline to a second image and compares the files **byte for byte** after a truncating checkpoint of the original. The seeded fuzz harness extends the same oracle to randomized workloads with restarts, torn journal tails, and image deletion.

3.5 The fallback that was not needed

The plan reserved a custom VFS (intercepting `xWrite` on the WAL) as fallback if public interfaces proved insufficient. They did not; the VFS remains a documented escape hatch should a future SQLite change the WAL contract.

4 Storage and recovery

4.1 The file set

```
data/
  LOCK                     exclusive process lock (flock)
  identity                 node id, database id, current configuration
  paxos-<config16hex>.log  framed, checksummed protocol journal (per epoch)
  payloads/aa/<62hex>      immutable frame payloads, named by SHA-256
  snapshots/<config>/      db image + manifest per sealed epoch
  snapshots/tmp-*          in-progress generations (crash debris, GC'd)
  CURRENT                  installed snapshot pointer
  current.db               materialized SQLite image (rebuildable)
```

Authority is unambiguous: **journal plus payloads plus snapshots** are the database. `current.db` is a cache of applying them; `current.db-wal` and `-shm` are working artifacts of the live connection and are deleted on every open before rebuild.

4.2 Ordering rules

The host enforces five write-ordering invariants, and the crash tests exist to catch any violation:

1. payload fsync and an explicit `payload_stored` ACK **before** a value-bearing promise, accept, or commit is released to that peer;
2. journal append + fsync **before** any dependent message leaves the process (sync-before-send) — structurally guaranteed because `consumeEffects` journals before it fills the outbox;
3. parent-directory sync after every authoritative create/link/rename, so journal names, payload objects, `identity`, `CURRENT`, and snapshot generations survive with their synced contents;
4. commit **before** apply, applied contiguously in slot order;
5. session-row update inside the captured transaction **before** acknowledgement (acknowledge-after-session-update).

4.3 Recovery sequence

`Node.open` performs, in order: lock the directory; load or create `identity`; open the payload store; open the epoch journal and replay it into `DurableState` (truncating a torn final record; rejecting interior corruption); restore the protocol node; validate the installed snapshot's identity, epoch relation, manifest fields, and image digest; resume an interrupted snapshot install if required; **(one-member only)** campaign; rebuild the materialized image by always deleting `current.db`/`-wal`/`-shm`, copying the snapshot base, then chain-validating and offline-applying every committed batch; complete a pending epoch rollover if a decided stop sign was replayed; finally validate that the image's recorded `batch_id` equals the last committed descriptor's.

> **Torn tail versus interior corruption**
>
> A record that fails to parse **and** touches end-of-file is a torn append: it is truncated and recovery proceeds — that vote or commit was never confirmed to anyone. A record that fails **inside** the valid prefix is corruption of state the node may have promised: the node refuses to open rather than vote with amnesia.

4.4 Snapshots and epoch rollover

An epoch holds at most 256 slots; the host checkpoints before the bound (reserving four slots so the stop sign always fits). A snapshot is a **decided** object:

1. materialize: truncating checkpoint on the leader's capture connection (followers are already fully materialized offline);

2. build `snapshots/tmp-<config>/` with the database copy and a manifest (database id, sealed configuration, applied slot, chain, db SHA-256), then rename it into place;
3. propose `checkpoint(metadata)` where the metadata is `zx1 <name> <manifest-sha256>` — the stop sign seals the epoch;
4. when the stop sign commits, every member verifies its local generation against the decided digest (followers build theirs from the offline image; byte-determinism makes the digests match), installs `CURRENT`, bumps `identity`, starts the next epoch's journal, and re-elects.

Rollover is crash-resumable at every step: the decided stop sign in the sealed journal replays on restart and the completion re-runs idempotently.

4.5 Garbage collection

After a rollover the node keeps: the new epoch's journal, the sealed epoch's journal (fallback generation), the two newest snapshot generations, and every payload referenced by a retained journal. Everything older is covered by the installed snapshot and deleted. A payload is never collected because of age or ballot change — only when no retained journal references it.

4.6 The journal format

Each record:

Offset/size	Field	Meaning
0 / 4	`magic`	`0x315a584a` ("ZXJ1"), little-endian
4 / 1	`version`	format version, 1
5 / 1	`kind`	write tag: promise 0, accept 1, commit 2
6 / 2	`reserved`	zero
8 / 8	`sequence`	strictly increasing from 1 per epoch
16 / 4	`payload_len`	encoded write length
20 / 4	`crc32`	over header-sans-crc plus payload
24 / n	`payload`	canonical little-endian `Write` encoding

Writes encode ballots (`round:u64, priority:u32, node:u32`), slots, and entries; an entry is a command descriptor or a stop sign (configuration id, members, metadata).

5 Role-aware clusters

5.1 Topology

`zaxon serve` hosts one node behind one TCP endpoint that carries both peer and client traffic (the first frame on any connection is a `hello` that says which). Voters dial every storage node; learners dial only voters so they can return payload ACKs and requests. Learner-to-learner links carry no protocol information and are omitted. A connection sends in one direction; inbound connections are receive-only. Gateways proxy client streams and never enter this topology. The result has unambiguous stream ordering per direction, automatic reconnect with backoff, and protocol repair (`reconnected`) on every re-establishment.

Threads are few and boring by design: one accept loop, one reader per connection, one sender per peer (owning a bounded frame queue), one tick thread driving elections, heartbeats, and retransmission. A single mutex guards the node; the write path's fsyncs serialize under it exactly as the ordering contract requires.

5.2 Registry, voter membership, and scale

All nodes are configured with the same role registry; the shared `database_id` is derived deterministically from the sorted voter ids (plus an optional `--cluster-id`), so a mis-configured process cannot join the wrong database — the `hello` handshake rejects it. Election priority is the node id, giving deterministic tie-breaking without affecting safety.

Only `data-voter` and `witness` nodes enter Paxos quorums. A data voter may campaign and materializes SQLite; a witness votes but cannot campaign or serve SQL. `standby` and `read-replica` are learners: they store the chosen log and materialize SQLite without sending promises or accepted votes. A standby is promotion-eligible by an operator; a read replica is not. A `gateway` stores nothing and routes end-to-end authenticated client connections. Every registry must contain at least one data voter; an all-witness quorum is safe but cannot make progress and is rejected at startup.

The first release bounds one Paxos group to one through nine voters so its fixed-size core state stays reviewable. The allocator-backed product registry may contain any runtime-sized number of learners and gateways subject to host resources. Paxos has no theorem limiting a deployment to three or nine nodes; the implementation bound is deliberate. Adding replicas improves read and failure placement, not the throughput of SQLite's single writer. Aggregate write scale requires independent databases or shards.

5.3 Payload gating: votes never outrun bytes

The correctness-critical transport rule: **a member never processes a value-bearing promise, accept, or commit whose payload bytes are not durable and digest-verified in its local store.**

1. The sender pushes `payload_data`; the receiver verifies, fsyncs, and atomically installs it, then returns `payload_stored`. Only that ACK releases matching envelopes from the sender's bounded per-peer gate.
2. If a receiver still lacks the payload (reconnect races, drops), it **holds** the envelope in a bounded queue, requests the payload by hash, and only steps the envelope after the bytes are stored and fsynced. Bounded means bounded: overflow drops the envelope and Paxos

retransmission recovers it.

Promises are gated because the core may complete Phase 1 and emit recovery accepts in the same `step` transition. Protocol v4 supports provider-file PSK challenge-response with a fresh nonce, a connection-unique session key, and a monotonically sequenced HMAC on every post-handshake frame. A wrong proof, replay, tampering, or version downgrade closes the stream. Without a provider, the server refuses non-loopback addresses. HMAC does not encrypt SQL; confidential deployments still need an encrypted tunnel.

Because a follower stores the payload before it votes, any chosen value has its bytes durable at a write quorum — the recovery path can always fetch by digest from some surviving voter, and a node missing a payload for a committed slot refuses to serve rather than invent state.

The leader separately sends `learner_commit(configuration, slot, entry)` to each non-voter only after the referenced payload has received that learner's durable storage ACK. This is a chosen-value certificate from a configured voter, never a vote by the learner. Learners reject other senders, epochs, conflicting duplicates, and non-contiguous application; they buffer only the compile-time-bounded reorder window. Reconnect resets the sender cursor and replays the chosen prefix idempotently. The certifying voter is also the learner's redirect hint for leader-only requests.

The leader emits a non-voting learner heartbeat every 20 ticks (500 ms with the default tick) with its current decided slot. A local `any` read may include `freshness_ms`; it is rejected if leader contact is older than that bound or if the heartbeat says the learner is behind. Without a freshness bound, `any` explicitly permits an arbitrarily stale local snapshot.

5.4 Commit, apply, and the follower image

Only the leader executes SQL. Followers learn commits and apply the payload pages offline to `current.db` — the same deterministic apply as recovery. When leadership changes, the old leader's live WAL may hold frames Paxos never decided (its in-flight write lost the slot); the host detects every such case in commit accounting, closes the capture connection, discards the WAL, and rebuilds the image from the decided log before serving again.

> **Epoch fencing on the wire**
>
> Every envelope frame carries the sender's configuration id. Frames from an older epoch are dropped; frames from a newer epoch mean this member missed a sealed rollover and triggers a snapshot transfer. The Paxos messages themselves never cross epochs.

5.5 Catch-up and snapshot transfer

Within an epoch, a lagging voter is repaired by the core protocol: reconnection sends `learn`, the leader resends missing commits, and the tick loop requests catch-up whenever heartbeats reveal a decided slot ahead of the local one. A learner is repaired by replaying voter-certified chosen entries. Across a sealed epoch the journal alone is not enough: the member requests a snapshot, the peer streams the installed generation (manifest, then the database image in 1-MiB chunks), and the receiver verifies the digest, installs `CURRENT` and `identity`, starts the new epoch's journal empty, rebuilds its image from the snapshot, and catches up the suffix normally. The cluster test exercises exactly this: a member stopped across a rollover rejoins and converges to byte-identical content.

5.6 Failpoints

The crash matrix is automated with process-killing failpoints (`_exit(137)`, no cleanup — a SIGKILL at a chosen line): before/after payload sync, after journal append, after journal sync, before follower apply, and before the client reply. A test controller arms them at runtime through an RPC that only servers started with `--enable-failpoints` honor. The cluster scenario's centerpiece kills the leader **after quorum choice, before the client reply**, retries the same session sequence at the new leader, and proves the write applied exactly once.

6 Consistency and the client contract

6.1 Write acknowledgement

A client write returns one of three outcomes, and the distinction is the contract:

1. **success**: the slot committed and the decided value at that slot is this client's `batch_id`. The transaction is durable at a quorum and applied.
2. **failure**: the SQL failed (rolled back, nothing replicated) or the session sequence was rejected — provably no side effect.
3. **ambiguous** (`ambiguous`, `timeout`, connection loss): the write may or may not have committed. The client retries the **same session sequence** — never a blind re-execution.

6.2 Exactly-once sessions

A session is a replicated row: `(id, next_sequence, last_sequence, last_changes, last_activity_slot)`. `exec` with `(session, sequence)` checks the row first:

1. `sequence == last_sequence`: return the recorded result (`replayed`);
2. `sequence == next_sequence`: execute; the row update rides inside the captured transaction, so result and data commit atomically;
3. `sequence < next - 1`: `ResultExpired`; `sequence > next`: `SequenceGap`; unknown id: `UnknownSession` — none execute SQL.

Because the session table travels with the frames, a retry lands correctly at a **new** leader after a crash: its image already contains the session row the old leader committed. The cluster failpoint test demonstrates the full loop.

Sessions are bounded. Every session write bumps a replicated activity counter (`write_seq`) and stamps the session; `expire-sessions --retain n` deletes sessions idle for more than the last `n` session writes, as a normal replicated write. Storage stays proportional to active sessions plus the result window, not to history.

6.3 Read levels

Level	Semantics	Cost
`any`	Local applied state of whichever member answered; may lag the leader. Labeled in the response.	none
`leader`	The leader's applied state. Serializable against the single writer; can be stale only across an unnoticed leadership change.	redirect
`linearizable`	Includes every write acknowledged before the read began, guaranteed current across leader changes.	one fence round

6.4 The quorum read fence

The linearizable path performs no log append and no disk sync:

1. the leader records its ballot b and fence slot $s = \text{decidedThrough}$;
2. it probes every peer: *"is b still exactly the ballot you have promised?"*;
3. each peer answers from its durable `promised` — an equality check, no writes;
4. the fence completes when a read quorum (leader plus one, for three voters) confirms b **and** the leader has applied through s; then the query runs on applied state.

Why this is safe. Ballots are totally ordered and a promise is monotone. For a competing write to have been acknowledged after our fence began, some higher ballot $b' > b$ must have completed a write quorum

of promises. Write and read quorums intersect, so some member of our fence quorum would already have promised b' — and its answer would have failed the equality check. Contrapositive: a completed fence at b proves no higher-ballot write completed before the probe, so applied-through-s state contains every acknowledged write. A fence also fails immediately if the leader's own ballot or role moves while it is outstanding.

> **The committed barrier stays as the oracle**
> The log still carries a `read_barrier` command: appending one and waiting for it is the slow, obviously correct reference read. The test suites use it as a differential oracle for the fence path; production reads use the fence.

6.5 The crash matrix

Crash point	Client outcome	Recovery oracle
Before payload sync	failure/unknown, never success	No vote references the incomplete payload; the store's temp-file install leaves no partial object.
After payload sync, before accept append	unknown	Payload may be orphaned garbage; GC collects it; nothing infers it chosen.
After accept append, before journal sync	unknown	Torn tail is truncated; the vote never claimed to survive.
After accept sync, before send	unknown	The vote survives replay and is available to a later leader.
After quorum choice, before client reply	unknown	Election recovery preserves and commits the chosen value; the session retry replays the recorded result.
After commit sync, before apply	unknown/delayed	Contiguous replay applies exactly once.
After apply, before reply	unknown	Same sequence returns the stored result; never applies twice.
During snapshot install	existing traffic unaffected	Restart picks a complete generation — old or new — never `tmp-*`.

The hooks and harnesses exist, but the current automated suite does not yet exercise every row in both one-node and three-node roles. It covers torn tails, stale/corrupt image replacement, both snapshot transition prefixes, and the cluster leader kill after choice/before reply. Completing the remaining rows and required schedule counts is a release blocker in the product plan.

7 Operations

7.1 Running one durable node

```
zaxon serve --data /var/lib/app --node 1 --listen 127.0.0.1:9901
```

or skip the server entirely and embed: every CLI command with `--data` opens the node in-process, exactly as a linking application would. The directory `flock` guarantees one host process at a time (a locked directory is exit code 4, never a corruption risk).

7.2 Running three voters

Give every member the **same** membership, its own id, directory, and endpoint:

```
zaxon serve --data /d/n1 --node 1 --listen 10.0.0.1:9901 \
    --peer 2@10.0.0.2:9901/data-voter \
    --peer 3@10.0.0.3:9901/data-voter
zaxon serve --data /d/n2 --node 2 --listen 10.0.0.2:9901 \
    --peer 1@10.0.0.1:9901/data-voter \
    --peer 3@10.0.0.3:9901/data-voter
zaxon serve --data /d/n3 --node 3 --listen 10.0.0.3:9901 \
    --peer 1@10.0.0.1:9901/data-voter \
    --peer 2@10.0.0.2:9901/data-voter
```

The shared database identity derives from the member ids (add `--cluster-id <text>` to separate otherwise-identical clusters). Wait for the cluster:

```
zaxon wait --connect 10.0.0.1:9901 --leader --timeout-ms 10000
zaxon leader --connect 10.0.0.1:9901
```

7.3 Non-voting and routing nodes

Every storage node receives the same bootstrap registry. A read replica uses `--role read-replica`, and each voter lists it as `--peer 4@10.0.0.4:9901/read-replica`. Use `standby` for a promotion-eligible copy, `witness` for a voting non-SQL failure domain, and `gateway` for a stateless client endpoint. An existing data directory pins its role; changing the CLI role without a controlled migration is rejected.

7.4 The command surface

Command	Purpose
`sql`	Interactive shell (embedded or connected); read statements route to query, everything else to exec.
`exec`	One replicated write transaction; `--session`/`--sequence` makes it idempotent.
`query`	Read-only; `--level any|leader|linearizable`; defaults to `linearizable`; local learners accept `--freshness-ms`; `--json`.
`session`	Open a replicated retry session.
`expire-sessions`	Delete sessions idle beyond `--retain` recent session writes.
`status`	Node id, node type, Paxos role, leader, ballot, decided/applied slots, configuration, chain, snapshot.
`wait`	Block until `--applied <slot>` and/or `--leader`.
`snapshot`	Materialize, seal the epoch, roll to the next one.
`backup`	`VACUUM INTO` a consistent logical copy — works on leader and follower alike.

Command	Purpose
`integrity-check`	SQLite `integrity_check` + descriptor chain + payload availability. Exit 3 on failure.
`stop`	Graceful server shutdown (client mode).

Exit codes: 0 ok · 1 SQL/session error · 2 usage · 3 integrity failure · 4 unavailable (locked, corrupt, no reachable leader).
Client mode accepts a comma-separated endpoint list and follows `not_leader` redirects automatically; scripts can point at all three members and ignore leadership entirely.

7.5 Failure playbook

Symptom	What happens / what to do
One member down	Writes and linearizable reads continue on the remaining two. Restart the member; it catches up automatically (journal suffix, or snapshot transfer across a sealed epoch).
Two members down	No quorum: writes and fenced reads refuse; `--level any` reads still serve locally. Add `--freshness-ms` to reject a disconnected or lagging learner instead. Restore a member.
`current.db` lost or restored from an old copy	Delete it (or leave the stale copy); the node rebuilds from snapshot plus journal and validates the batch marker. No operator action beyond restart.
Journal tail torn by power loss	Truncated automatically on open; the lost suffix was never acknowledged.
Journal corrupt in the middle	The node refuses to open. Recover by reimaging from a healthy member: empty the directory and restart — snapshot transfer plus catch-up rebuild everything.
Disaster recovery of last resort	`zaxon backup --to app.db` on any surviving member yields a plain SQLite file usable anywhere.

7.6 Monitoring

`status --json` is the machine surface: `node_type`, Paxos `role`, current leader, ballot, `decided_slot` versus `applied_slot` (lag), journal record count, epoch capacity, chain hash (compare across members: equality means identical applied history), and the installed snapshot generation. `members --json` returns the runtime registry with role, voter/campaign/read/ write/promotion capabilities, self identity, and the current leader hint.

8 Embedding APIs

8.1 The Zig library

Add the package and import `zaxonlite`; the parent `paxos` library and the pinned SQLite amalgamation come with it.

```
const zaxonlite = @import("zaxonlite");

var node = try zaxonlite.Node.open(gpa, io, .{ .directory = "./data" });
defer node.close();

_ = try node.exec("create table items(id integer primary key, v text)");

const session = try node.openSession();
_ = try node.execIdempotent(session, 1,
    "insert into items(v) values ('tea')");

var rows = try node.query(gpa, "select id, v from items order by id");
defer rows.deinit();

try node.snapshot();                       // seal the epoch online
const report = try node.integrityCheck();  // sqlite + chain + payloads
```

Key surface on `Node`:

Function	Contract
`open` / `close`	Owns the directory; `OpenOptions` selects one-member or cluster membership, election priority, and a fixed database identity for clusters.
`exec`	One replicated write transaction. In a one-member configuration the result is committed and applied on return.
`openSession` / `execIdempotent` / `expireSessions`	Exactly-once retry sessions (chapter 6).
`query`	Read-only, arena-backed result set; uses the live connection on the leader, a short-lived one elsewhere.
`snapshot`	Online checkpoint + epoch seal + rollover (one-member); cluster hosts use

Function	Contract
	`prepareCheckpoint` and complete on decision.
`backup` / `contentHash` / `integrityCheck` / `status`	Operations surface, identical to the CLI's.
`stepEnvelope` / `tickProtocol` / `peerReconnected` / `requestCatchUp` / outbox	The transport-host surface `server.zig` is built on — available to embedders who bring their own transport.

For a transport-owning member, use `Embedded.open` with a runtime slice of `EmbeddedMember { id, address, role }`. The facade copies the registry, owns the listener, peer senders, tick loop, and client routing, and exposes `exec`, `query`, and generic JSON `call`. At most nine members may be voting roles; learners do not consume those slots.

8.2 The C ABI

`libzaxonlite.a` plus `zaxonlite.h` export the embedded surface with CLI-aligned return codes (0 ok, 1 SQL/session, 2 misuse, 3 integrity, 4 unavailable):

```c
#include <zaxonlite.h>

zaxonlite *db = NULL;
if (zaxonlite_open("./data", &db) != 0) return 1;

int64_t changes;
zaxonlite_exec(db, "create table t(a integer primary key, b text)",
               &changes);

uint64_t session;
zaxonlite_session_open(db, &session);
bool replayed;
zaxonlite_exec_idempotent(db, session, 1,
    "insert into t(b) values ('x')", &changes, &replayed);

char *json;
if (zaxonlite_query_json(db, "select * from t", &json) == 0) {
    puts(json);              /* {"columns":[...],"rows":[[...]]} */
    zaxonlite_free(json);
}

zaxonlite_snapshot(db);
zaxonlite_integrity_check(db);
zaxonlite_close(db);
```

The matching cluster C surface is `zaxonlite_cluster_open` with `zaxonlite_cluster_options` and `zaxonlite_member` records. It owns transport and exposes `zaxonlite_cluster_exec`, `zaxonlite_cluster_query_json`, and `zaxonlite_cluster_call_json`. Member strings and the registry are copied before open returns.

Each handle owns an internal event-loop instance; use a handle from one thread at a time (SQLite connection discipline). `zaxonlite_last_error` returns the most recent message for the handle.

8.3 The client RPC protocol

Anything that can frame bytes over TCP can be a client: one `hello` frame (`kind = client`), then one JSON request per `rpc_request` frame and one JSON response per `rpc_response`. Requests are objects with an `op` — `exec`, `query`, `session`, `wait`, `status`, `leader`, `members`, `snapshot`, `backup`, `integrity`, `expire-sessions`, `stop` — mirroring the CLI exactly; error responses carry `{"ok":false,"error":code,` and, for `not_leader`, the leader's endpoint for redirect-following. An `any` query may carry `freshness_ms`; a learner rejects it when leader contact exceeds the bound or its applied slot trails the latest reported decision.

9 Format reference

All integers are little-endian unless noted. Hashes are SHA-256.

9.1 Payload ("ZXPL")

The immutable object named by `payload_hash`:

Offset/size	Field	Meaning
0 / 4	`magic`	`0x4c50585a` ("ZXPL")
4 / 1	`version`	1
5 / 3	reserved	zero
8 / 4	`page_size`	power of two, 512–65536
12 / 16	`database_id`	must match the node's identity
28 / 4	`transaction_count`	≥ 1
32 / 4	`frame_count`	≥ 1

followed by `transaction_count` records of 32 bytes (`session_id:u64, sequence:u64, first_frame:u32, frame_count:u32,`
`change_count:i64`), then `frame_count` records of 8 bytes (`page_number:u32, commit_size:u32` — big-endian values already converted to native descriptors), then the raw page images. Validation requires transactions to tile the frame range in order and each to end on a commit frame.

9.2 Snapshot manifest

Plain `key=value` lines, hashed whole for the stop-sign digest:

```
format=1
database_id=<32 hex>
sealed_configuration_id=<decimal>
applied_slot=<decimal>          # slot before the stop sign
chain=<64 hex>
db_sha256=<64 hex>
```

The stop-sign metadata is `zx1 <snapshot-name-16hex> <manifest-sha256>`; a generation is only installable when its manifest hashes to the decided value.

9.3 Identity file

```
format=2
node_id=<decimal>
database_id=<32 hex>
configuration_id=<decimal>
role=<data-voter|witness|standby|read-replica>
```

Format 1 omitted `role` and is read as `data-voter`; a later identity write upgrades it to format 2. Opening a directory under another role is rejected. This prevents a restart from silently changing a voter into a learner or the reverse. Gateways have no identity file.

9.4 Wire frames

Every connection: `u32 total_len` (body length + 1), `u8 kind`, body. Bodies over 64 MiB are protocol errors.

Kind	Name	Body
1	`hello`	`version:u16, kind:u8 (0 peer, 1 client), node_id:u32, database_id:u128, configuration_id:u64`
2	`envelope`	`configuration_id:u64`, then the encoded Paxos envelope (`from:u32, to:u32, tag:u8`, message fields)
3	`payload_data`	`hash:[32]u8`, payload bytes (receiver verifies the hash)
4	`payload_request`	`hash:[32]u8`
5	`fence_request`	`ballot (u64,u32,u32), fence_id:u64, fence_slot:u32`
6	`fence_ack`	`fence_id:u64, ok:u8, promised ballot`
7	`snapshot_request`	empty
8	`snapshot_begin`	`configuration_id:u64, name:[16]u8, db_size:u64, manifest_len:u32, manifest`
9	`snapshot_chunk`	`offset:u64`, image bytes
10	`snapshot_end`	empty
11 / 12	`rpc_request` / `rpc_response`	one JSON object
13	`payload_stored`	`hash:[32]u8`; emitted only after verified durable installation
14 / 15	`auth_challenge` / `auth_response`	fresh nonce and mutual HMAC-SHA256 proofs
16	`backup_begin`	`size:u64, sha256:[32]u8`
17	`backup_chunk`	`offset:u64`, backup bytes
18	`backup_end`	empty
19	`learner_commit`	`configuration_id:u64, slot:u32`, then one canonical chosen entry; accepted only from a configured voter
20	`learner_heartbeat`	`configuration_id:u64, decided_through:u32`; drives bounded-staleness checks but carries no vote

The current hello version is 4. Version 2 added the storage-ACK gate and applied payload materialization to value-bearing Phase-1 promises. Version 3 added mutual authentication and backup streaming. Version 4 adds durable, voter-certified chosen-entry delivery and freshness heartbeats to non-voting learners. After authentication, every application body is wrapped as `sequence:u64 || body || hmac:[32]u8`; exact next-sequence validation rejects replay. Older versions are rejected rather than silently downgraded. Envelope message tags: prepare 0, promise 1, promise_done 2, accept 3, accepted 4, commit 5, learn 6, nack 7, heartbeat 8 — carrying exactly the fields of the core protocol's message types, with entries encoded by the same canonical codec the journal uses.

9.5 The replicated command encoding

`Command` encodes into a fixed 149-byte canonical form (tag byte, then fields, zero padding enforced on decode): tag 0 `noop`, tag 1 `transaction_batch` (all descriptor fields in order), tag 2 `read_barrier` (nonce). Non-canonical padding or unknown tags are decode errors — one byte pattern, one meaning.

10 Verification

10.1 Test layers

Layer	Focus	Command
Unit	Codecs, chain hashes, journal replay/torn-tail/corruption, payload store faults, WAL capture/apply byte-identity spike, wire round trips.	`zig build test`
Single-process integration	Restart recovery, journal-authoritative rebuild, stale-image convergence, idempotent sessions, session expiry, snapshot rollover (explicit and capacity-triggered), failed SQL isolation, locking.	`zig build test-single`
Three-process cluster	Election, writes through every endpoint, follower stop/catch-up, hash comparison, leader SIGKILL failpoint with exactly-once retry, rollover with a stopped member (snapshot transfer), image rebuild, total restart.	`zig build test-cluster`
Role cluster	Three data voters, a non-campaigning witness, standby and read-replica chosen-log catch-up, role read restrictions, leader hints, and bounded-freshness refusal after voter disconnection.	`zig build test-roles`
Gateway	Authenticated RPC passes end to end through a process with no Paxos or SQLite state.	`zig build test-gateway`

Layer	Focus	Command
Adverse network/storage	Real TCP loss, semantic duplication, pair reordering, seven-byte frame fragmentation, and delayed durable sync.	`zig build test-fault-network`
CLI contract	Exit codes, JSON shapes, session semantics, scripted shell, locking, offline client mode.	`zig build test-cli`
C ABI	Every exported function, including locking and replay-after-reopen.	`zig build test-cabi`
Fuzz	Seeded: decoder robustness (random + mutated-valid bytes), journal-file damage, end-to-end random SQL with crashes and rebuild-convergence oracle.	`zig build fuzz`
Soak	Sustained mixed load with a live row-count model, replay checks, snapshots, restarts.	`zig build soak`
Benchmark	Write/read latency percentiles, recovery and rebuild times, ReleaseFast.	`zig build benchmark`
dqlite comparison	Pinned three-voter durable state, equal one-row workload and payload, warmup exclusion, verification, JSON output.	`benchmarks/compare-dqlite-3node.sh` (Linux)
rqlite comparison	Three voters using installed binaries, equal one-row workload and payload, no queued writes, strong verification, CLI membership evidence, percentile latency, JSON output.	`benchmarks/compare-rqlite-3node.sh`

`zig build test-cluster -Dcluster-runs=N` repeats the full scenario for flake hunting; 100 consecutive runs are an explicitly deferred CI stress target, not a result claimed by this release.

10.2 The mandatory cluster scenario

The controller spawns three real `zaxon serve` processes on loopback ports and drives them over the public RPC protocol — no test back door into node internals. Every wait is deadline-based on observable conditions (status fields), and a failure dumps all three statuses and log tails. The script: elect; create schema; submit requests 1–100 through all three endpoints (half through a session); verify a linearizable count; stop a follower; write 101–150; restart and wait for catch-up; compare chain and content digests on all three; arm the leader's `before_client_reply` failpoint and submit a session write — the leader dies after quorum choice; retry the same sequence at the new leader and prove it applied exactly once; roll the epoch with another member stopped and watch it rejoin via snapshot transfer; delete a member's `current.db` and watch it rebuild; kill everything without ceremony; restart all; verify 153 rows, the failpoint row exactly once, clean integrity, and identical digests.

10.3 Persistence assertions, mapped

Chapter 6's crash matrix and the plan's persistence assertions each name their enforcement: chain equality across members (cluster digests), contiguous monotone apply (commit accounting), acknowledged writes surviving single-node loss and total restart (cluster scenario), at-most-once session sequences (single, CLI, C ABI, cluster failpoint), payload-before-vote (transport gating plus store fault tests), sync-before-send (structural: journal fsync precedes the outbox), refusal on missing durable state (corrupt-journal and missing-payload tests), torn-tail-versus-interior discrimination (journal tests and fuzz), and quorum loss blocking acknowledgement (two-members-down refusal).

10.4 Measured baselines

On the development machine (Apple Silicon, Debug-mode servers except where noted), single-node ReleaseFast:

```
write     1000 ops     672 ms    1485 ops/s   p50 617 us  p95 729 us  p99 818 us
read     10000 ops      21 ms  462490 ops/s   p50   2 us  p99   2 us
recovery  1000 committed writes replayed + validated in 11 ms
rebuild   image restored from snapshot in <1 ms
```

Each write is a full pipeline: SQLite transaction, frame capture, payload fsync, journal append, journal fsync. The read path performs no consensus append and no disk sync — that is the fence path's cost model, plus one network round trip in a cluster. Numbers are baselines, not claims; the benchmark harness prints its own on your hardware.

The first installed-rqlite comparison on the same Apple Silicon development host used rqlite v10.2.7 / SQLite 3.53.2, three voters, 100 excluded warmup writes, then 1,000 measured 256-byte autocommits. One observed run was:

```
zaxonlite   474.6 ops/s   p50 1.69 ms   p95 1.97 ms   p99 2.32 ms
rqlite       45.0 ops/s   p50 21.96 ms  p95 27.96 ms  p99 33.04 ms
```

The installed `rqlite` CLI confirmed three reachable voters and 1,000 measured rows; the driver also checked a strong count and exact-payload predicate. This is one local end-to-end observation, not a portable claim or a consensus-only microbenchmark. dqlite execution remains deferred to a supported Linux host. The raw JSON is retained under `benchmarks/results/`.

10.5 Real-world failure and recovery comparison

The second product comparison is a deterministic order-processing simulation, not a single-row write loop. Each product runs alone as three voters with 1,000 customers, 500 products and four clients distributed across all endpoints. The traffic is 70% reads (inventory, customer history and sales dashboards) and 30% orders. An order is one SQLite transaction whose trigger creates an order line, decrements inventory and records an accounting-ledger entry.

Measured reads use each product's quorum-fence `linearizable` level. rqlite's `strong` mode is reserved for the final verification barrier because rqlite's read-consistency guide describes that mode as a testing tool which writes through Raft; it recommends `linearizable` when that guarantee is required in production. Every write carries a unique operation ID and uses `insert or ignore`, so a client can safely retry an ambiguous response after a process dies.

After 100 warmup operations the controller runs three 400-operation phases:

1. healthy three-voter traffic;
2. `SIGKILL` one follower, continue with a quorum, restart it and wait until its local SQLite copy matches;
3. `SIGKILL` the leader, begin traffic immediately, retry through election, restart the old leader and wait for local catch-up.

It then kills all three processes, restarts their original identities and data directories, and checks every local copy. Two consecutive full runs on the same Darwin-arm64 development host produced these ranges:

Measurement	**Zaxonlite**	**rqlite v10.2.7**
Healthy mixed throughput	1,783-1,796 ops/s	170-177 ops/s
One-follower mixed throughput	1,045-1,073 ops/s	214-222 ops/s
Leader crash to first success	591-597 ms	2,190-2,402 ms
Restarted follower catch-up	112-166 ms	841-949 ms
Restarted old-leader catch-up	175-338 ms	548-554 ms
Total three-node restart	446-457 ms	1,294-1,627 ms

In the retained run, healthy all-operation p50/p95 latency was 1.63/5.58 ms for Zaxonlite and 15.26/68.58 ms for rqlite. The leader outage appears honestly in tails: linearizable-read p99 was 602 ms and 2,411 ms respectively. Both systems finished with exactly 372 orders, lines, ledger records and operation IDs; 942 inventory units; 4,428,073 cents revenue; no negative stock; identical values on all three nodes; and clean SQLite integrity. Zaxonlite's chain and payload-store checks also passed.

These are local end-to-end observations, not a proof that Paxos is faster than Raft. Front ends, storage layouts, election timers and implementations differ. The initial rqlite bootstrap time is intentionally omitted because the manual join path incurred its default three-second retry interval. This schedule tests process loss and restart, not network partition or disk corruption; those are covered by the separate adverse-network and crash suites. Reproduce with `benchmarks/compare-rqlite-realworld-3node.py`; the full JSON is `benchmarks/results/realworld-rqlite-v10.2.7-darwin-arm64-2026-07-20.json`.

10.6 What remains beyond this book

Engineering headroom, tracked in the product plan: pipelined/batched writes under one chosen value (the descriptor already carries `transaction_count`), group fsync, longer random fault schedules beyond the deterministic adverse run, automatic voter replacement, and sharding. The 10,000-crash, 100-run, and 1-GiB gates are explicitly deferred; the checked large recovery fixture is 1 MiB.